Week No. 3

Lab Manual: Information Security

Lab Title: Classical Ciphers (Code)

Lab Objectives

By the end of this lab, students will be able to:

4. Perform cryptanalysis on classical substitution ciphers.
5. Use frequency analysis to identify cipher patterns.
6. Demonstrate the process of recovering plaintext without knowing the key (Brute force).

Learning Outcomes (CLOs & PLOs Mapping)

CLO	CLO Statement	Bloom's Level	PLO
1	Apply Information Security techniques into practice.	P3 (Apply)	PLO-5

Lab Overview

This lab introduces **Cryptanalysis**, the art and science of breaking ciphers without knowing the key. Students will practically explore **frequency analysis**, a statistical technique used to infer key or plaintext patterns from ciphertexts of monoalphabetic ciphers such as Caesar.

In simple words: in Week 1 you were the sender who locks a message with a key. In Week 2 you are the code breaker who must open the lock without the key.

Quick Recap of Week 1

You only need these four ideas from Week 1. Everything else is explained again below where it is needed.

Idea	Reminder
Letter numbers	A=0, B=1, C=2 ... Z=25.
Caesar encryption	C = (P + K) mod 26. Every letter moves K steps forward.
Caesar decryption	P = (C − K) mod 26. Every letter moves K steps back.
Wrap-around (mod 26)	If a number goes above 25 subtract 26; if it goes below 0 add 26. Example: −3 + 26 = 23.

Example: HELLO with key 3 gives KHOOR. Notice both L's became O. **The same plaintext letter always becomes the same ciphertext letter.** This single fact is what makes today's attacks work.

New Words for Week 2

Term	Meaning
Cryptanalysis	Breaking a cipher without knowing the key.

Term	Meaning
Monoalphabetic cipher	One fixed letter-to-letter replacement for the whole message (Caesar and simple substitution are examples).
Frequency analysis	Counting how often each letter appears and comparing with normal English.
Brute force	Trying every possible key until the result makes sense.
Key space	The total number of possible keys. Caesar has 25 useful keys (26 with key 0).

Experiment 1: Understanding Frequency Distribution

Theory

Every language has a characteristic letter frequency. For example, in English, the letters E, T, and A appear most frequently.

What does this mean? If you take a long English text and count its letters, about 12 out of every 100 letters will be E, about 9 will be T, and so on. Rare letters such as Z and Q appear less than once in 100. This pattern is almost the same for every long English text (a novel, a news report or an email).

English letter frequency (%)

Letter	E	T	A	O	I	N	S	H	R	D	L	U
Frequency (%)	12.7	9.1	8.2	7.5	7.0	6.7	6.3	6.1	6.0	4.3	4.0	2.8

*The first six values (E to N) are the ones given in the lab. The extra six letters (S, H, R, D, L, U) are also very common. Together the order **ETAOINSHRDLU** is the famous "most common letters" list.*

How to Find Letter Frequency (Step by Step)

1. **Take the text.** Ignore spaces, digits and punctuation. Only letters count.
2. **Make all letters capital** so that "e" and "E" are counted together.
3. **Count** how many times each letter A to Z appears.
4. **Find the total** number of letters (add all counts).
5. **Calculate the percentage** of each letter:

Percentage formula
Frequency % = (count of the letter ÷ total number of letters) × 100

Worked Example: HELLO WORLD

Step 1: Remove the space: HELLOWORLD. **Step 2:** Count the letters: H E L L O W O R L D, so the total is **10**.

Letter	D	E	H	L	O	R	W
Count	1	1	1	3	2	1	1
Calculation	1÷10×100	1÷10×100	1÷10×100	3÷10×100	2÷10×100	1÷10×100	1÷10×100
Frequency %	10%	10%	10%	30%	20%	10%	10%

Check: all percentages add up to 100%.

Important lesson
In HELLO WORLD the top letter is **L (30%)**, not E. This is because the text is only 10 letters long. Frequency analysis needs **enough text** (ideally 100+ letters) to match the English table.

Practice Questions (Solve on Paper)

Q1. Count the letters in **MEET ME AT THE TOWER**. Write the count and percentage of every letter that appears (ignore spaces).

Answer:

Answer:

Q2. Make a frequency table for **ATTACK AT DAWN**. Which letter is the most frequent and what is its percentage? (Round to 1 decimal place.)

Answer:

Answer:

Q3. Why must spaces and punctuation be ignored when calculating percentages?

Answer:

Q4. Why is a frequency table of a 10-letter message less reliable than that of a 1000-letter message?

Answer:

Pseudo Code

```
INPUT: Plaintext
INITIALIZE dictionary for A-Z counts
FOR each character in Plaintext:
    IF character is alphabet:
        Convert to uppercase
        Increment count in dictionary
CALCULATE percentage frequency of each letter
DISPLAY frequency table
```

- **Expected Output:** A table showing each letter's frequency percentage.

Task

- Take a sample text (plaintext).
- Count the occurrence of each alphabet letter.
- Plot or print the frequency table.

Experiment 2: Frequency Analysis on Caesar Cipher

Theory

Since Caesar Cipher is a simple shift cipher, the frequency distribution of the ciphertext matches that of plaintext, only shifted. By identifying the most frequent letter in ciphertext and comparing it to expected frequencies (like 'E'), the shift key can be deduced.

Why this works (simple idea): In a Caesar cipher every E becomes the same letter, for example H if the key is 3. Because E is the most common letter in English, the most common letter in the ciphertext is very likely the disguised E. If we know that this letter stands for E, we can measure how far it moved. That distance is the key.

Key estimation formula
key = (cipher_letter − 'E') mod 26 (use letter numbers: A=0 ... Z=25)

Steps to Break a Caesar Cipher

1. Count the frequency of each letter in the ciphertext.
2. Find the most frequent letter.
3. Assume it stands for E. Calculate key = (cipher letter number − 4) mod 26. (E has number 4.)
4. Decrypt the whole ciphertext using that key (P = (C − key) mod 26).
5. Read the result. If it is meaningful English, you are done. If it is gibberish, go back to step 2 and try the **next** most frequent letter (or assume it stands for T).

Worked Example A: The Method Works

Ciphertext: **BJ BNQQ RJJY YMJ YJFR YTRTWWTB**

Step 1: Count. Total letters = 25. The counts are: J = 5, Y = 4, B = 3, R = 3, T = 3, and smaller counts for the rest.

Step 2: Most frequent letter = J (5 times).

Step 3: Assume J = E. J has number 9 and E has number 4, so key = (9 − 4) mod 26 = **5**.

Step 4: Decrypt with key 5. Example for the first word: B(1) − 5 = −4, add 26 gives 22 = **W**. J(9) − 5 = 4 = **E**. So BJ becomes WE.

Decrypting every letter gives: **WE WILL MEET THE TEAM TOMORROW**. It is meaningful English, so the key 5 is correct.

Worked Example B: When the Top Letter Fails

Ciphertext: **KHOOR ZRUOG** (only 10 letters)

Letter	O	R	K	H	Z	U	G
Count	3	2	1	1	1	1	1

Top letter is O. If O = E, key = (14 − 4) mod 26 = **10**. Decrypting with key 10 gives **AXEEH PHKEW**, which is gibberish. The text is too short, so the top letter is not the real E.

What now? Try the next candidate (R = E gives key 13, still gibberish) or use brute force (Experiment 4). Trying key 3 gives **HELLO WORLD**. Lesson: frequency analysis is a good first guess, not a guarantee.

Watch out for ties
If two letters have the same highest count, you cannot tell which one is E. **Try both** and see which key gives readable English.

How to Attempt the Lab Task

- Write the ciphertext without spaces and count each letter.
- Look at the top letters. Check whether more than one letter shares the highest count.
- For every candidate, calculate the key and decrypt the full text.
- The correct key is the one that gives readable English words.

Practice Questions (Solve on Paper)

Q1. The most frequent letter in a Caesar ciphertext is **H**. Assuming it stands for E, find the key.

Answer:

Q2. The most frequent letter in a ciphertext is **B**. Assuming it stands for E, find the key. (Be careful: the result is negative, so apply the wrap-around rule.)

Answer:

Q3. Use frequency analysis to break: **VNNC VN JC CQN URKAJAH**. Show the letter counts, the key and the plaintext.

Answer:

Answer:

Q4. Use frequency analysis to break: **WII XLI XIEGLIV XSQSVVSA**. Show the key and the plaintext.

Answer:

Answer:

Q5. Why can this method not easily break the Vigenère cipher from Week 1?

Answer:

Pseudo Code

```
INPUT: Ciphertext
CALCULATE frequency of each letter
IDENTIFY most frequent letter (assume it represents 'E')
ESTIMATE key = (cipher_letter - 'E') mod 26
DECRYPT using estimated key
OUTPUT possible plaintext
```

Task

- Ciphertext: "SOJZKXS KDGS OY ZUU KGYE"
- Perform frequency analysis to find the most probable key and recover plaintext.

Experiment 3: Frequency Analysis on Substitution Cipher

Theory

A simple substitution cipher maps each plaintext letter to a unique ciphertext letter. Frequency analysis helps guess the mapping by comparing observed frequencies to English frequency charts.

Difference from Caesar: Caesar moves every letter by the same amount, so only one number (the key) must be found. A general substitution cipher uses a **scrambled alphabet**, so every letter can map to any other letter. Example key (keyboard order):

Plain	A	B	C	D	E	F	G	H	I	J	K	L	M
Cipher	Q	W	E	R	T	Y	U	I	O	P	A	S	D

Plain	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Cipher	F	G	H	J	K	L	Z	X	C	V	B	N	M

Encrypting HELLO with this table: H→I, E→T, L→S, L→S, O→G, so the ciphertext is **ITSSG**. To decrypt, look up the cipher letter in the bottom row and read the plain letter above it.

Why not brute force? There are 26! (about 4×10^{26}) possible scrambled alphabets. Even a computer testing a billion keys per second would need about 13 billion years. So we use **frequency analysis** instead, which uses the language's patterns.

Steps to Break a Substitution Cipher

1. **Count** the frequency of every ciphertext letter and sort them from highest to lowest.
2. **Compare** with the English order ETAOINSHRDLU. Match the most frequent cipher letter to E, the next to T, then A, and so on. These are only guesses.
3. **Look for small words.** A repeated 3-letter word is probably THE. A single-letter word is A or I. A word like “?EET” suggests MEET.
4. **Apply the guessed mapping** to the ciphertext. Write the known letters and leave unknown letters as dashes or the cipher letter.
5. **Improve:** use the partly readable words to guess more letters. Repeat until the text is readable.

Worked Example

Ciphertext (keyboard-order key above): **ZIT ZTQD VOSS DTTZ ZIT ZTQEITK ZITKT**

Step 1: Count. T = 9, Z = 6, I = 4, Q = 2, D = 2, S = 2, others fewer.

Step 2: Match with English. Most frequent T → **E**. Second most frequent Z → **T**. (Remember these are guesses.)

Step 3: Find small words. ZIT appears three times and ends in T (= E). Pattern “T ? E” suggests **THE**. So Z = T (matches the guess in step 2) and **I = H**.

Step 4: Substitute. The text now reads: THE TE?? ?E?? ... Let us fill the gaps:

Cipher word	Partial text	Guess	New letters learned
ZIT	T H E	THE	Z=T, I=H, T=E
ZTQD	T E ? ?	TEAM	Q=A, D=M

Cipher word	Partial text	Guess	New letters learned
VOSS	? ? ? ?	WILL	V=W, O=I, S=L (double letter at the end)
DTTZ	M E E T	MEET	confirms D=M
ZTQEITK	T E A ? H E ?	TEACHER	E=C, K=R
ZITKT	T H E ? E	THERE	confirms K=R

Result: THE TEAM WILL MEET THE TEACHER THERE.

> **Remember**
> Frequency order is only a starting point. If a word does not make sense, change the guess and try again. Cryptanalysis is partly detective work.

How to Attempt the Lab Task

- Count the letters of the ciphertext and sort them by frequency (the hint says try the top letter as E).
- If two letters have the same count, test each one.
- Write the mapping you find as a table (cipher letter → plain letter) and apply it to every word.
- Check the result. If some words look slightly wrong, say so in your answer: the task only asks for a **partial** decryption.

Practice Questions (Solve on Paper)

Q1. Using the keyboard-order key above, encrypt **HELLO**. (Done as example in the theory; now encrypt **SEE YOU SOON** by yourself.)

Answer:

Q2. Using the same table, decrypt **LTT NGX**.

Answer:

Q3. In a ciphertext, the three most frequent letters are **X (14 times), Q (9 times), M (8 times)**. Write the first guesses for their plaintext letters.

Answer:

Q4. Why is brute force impossible on a general substitution cipher, yet frequency analysis works? Give two sentences.

Answer:

Answer:

Pseudo Code

```
INPUT: Ciphertext
CALCULATE frequency of letters in ciphertext
SORT letters by frequency (descending)
COMPARE with English frequency order (ETAOINSHRDLU...)
CREATE possible substitution mapping
APPLY mapping to ciphertext
OUTPUT partial or complete plaintext
```

Task

- Use the following ciphertext and attempt partial decryption:
- ZOLSS AOL JHU KPFF TVYF AOHA AOPZ PZ H ZLTWSL JYPWALY
- (Hint: Try matching most frequent ciphertext letter to ‘E’.)

Experiment 4: Automated Frequency Counter

Objective: Write a small Python/C++ function to automatically count letter frequency and suggest possible Caesar shifts.

Theory: Brute Force

Brute force means **trying every possible key**. A Caesar cipher has only 26 keys (0 to 25), so a computer can try all of them in a fraction of a second and print every result. A human then looks for the one line that reads as English. Frequency analysis helps by pointing to the most likely key first.

Worked Example: Brute Force on KHOOR

Key	0	1	2	3	4
Decrypted	KHOOR	JGNNQ	IFMMP	HELLO	GDKKN

Only key 3 gives a real word, so the plaintext is **HELLO**.

Understanding the Code Line by Line

The brute force loop in this lab is written like this. Let us understand every part:

Code	What it does
for key in range(26):	Repeats the work for key = 0, 1, 2 ... 25.
decrypted = ""	Starts with an empty answer for this key.
for char in ciphertext:	Looks at the ciphertext one character at a time.
if char.isalpha():	Checks that the character is a letter (not a space or symbol).
ord(char) - 65	ord() gives the ASCII number (A=65). Subtracting 65 turns A..Z into 0..25.
... - key	Shifts the letter back by the key (decryption).
% 26	Wraps around so the result stays between 0 and 25.
+ 65 and chr(...)	Turns the number back into a capital letter.
else: decrypted += char	Keeps spaces and punctuation unchanged.
print("Key:", key, "=>", decrypted)	Shows the key and the result so you can check whether it is readable.

Trace one letter: char = ‘K’, key = 3. ord(‘K’) = 75, 75 − 65 = 10, 10 − 3 = 7, 7 % 26 = 7, 7 + 65 = 72, chr(72) = **‘H’**.

A negative case: char = ‘B’, key = 3. 66 − 65 = 1, 1 − 3 = −2. In Python −2 % 26 = 24, then 24 + 65 = 89, chr(89) = **‘Y’**.

C++ difference
In C++ the % operator gives a **negative** answer for negative numbers, so write (ch - 'A' - key + 26) % 26. Adding 26 first keeps the number positive.

How to Use Brute Force with Frequency Intuition

1. Run the brute force loop and print all 26 results.
2. Scan for lines that look like English: common words such as THE, AND, IS, TO, OF.
3. Use frequency intuition to save time: the key suggested by the most frequent letter (as E) is the first one to check.
4. Confirm that the whole line is readable, not only one word.

Practice Questions (Solve on Paper)

Q1. Brute force the ciphertext **DWWDFN** for keys 0, 1, 2 and 3. Write each result and say which key is correct.

Answer:

Answer:

Q2. Using the formula chr(((ord(char) - 65 - key) % 26) + 65), calculate the output for char = 'B' and key = 5. Show every step.

Answer:

Answer:

Q3. How many times does the outer loop run? If the ciphertext has 6 letters, how many letter decryptions are done in total?

Answer:

Q4. What would happen to the space in "KHOOR ZRUOG" if the else branch was removed?

Answer:

Pseudo Code

```
for key in range(26):
    decrypted = ""
    for char in ciphertext:
        if char.isalpha():
            decrypted += chr(((ord(char) - 65 - key) % 26) + 65)
        else:
            decrypted += char
    print("Key:", key, "=>", decrypted)
```

Task

- Use brute-force + frequency intuition to find the correct decryption.

Coding Practice Questions

Coding Practice 1 (Solution in C++)

Problem: Write a C++ program that (a) reads a ciphertext from the user, (b) counts and prints the frequency of every letter, (c) finds the most frequent letter, assumes it is E and estimates the Caesar key, (d) shows the decrypted text, and (e) prints all 26 brute-force results. If two letters tie for the top spot, show every candidate.

Plan before coding:

1. Store the 26 letter counts in an array: counts[0] is A, counts[1] is B, and so on.
2. For each letter use ch − 'A' as the array position.
3. Percentage = count × 100.0 ÷ total letters (use 100.0 so C++ does not round to a whole number).
4. key = (position of top letter − position of E + 26) mod 26.
5. Reuse one function, caesarDecrypt(text, key), for both the estimated key and the brute-force loop.

C++ Code

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

// Shift every letter back by 'key'; other characters stay the same
string decrypt(string text, int key) {
    for (int i = 0; i < text.length(); i++) {
        if (isalpha(text[i])) {
            char ch = toupper(text[i]);
            text[i] = (ch - 'A' - key + 26) % 26 + 'A';
        }
    }
    return text;
}

int main() {
    string cipher;
    cout << "Enter ciphertext: ";
    getline(cin, cipher);

    // 1. Count each letter (counts[0] = A, counts[1] = B, ...)
    int counts[26] = {0};
    int total = 0;
    for (int i = 0; i < cipher.length(); i++) {
        if (isalpha(cipher[i])) {
            counts[toupper(cipher[i]) - 'A']++;
            total++;
        }
    }

    // 2. Print the frequency table
    for (int i = 0; i < 26; i++) {
```

```
        if (counts[i] > 0)
            cout << char('A' + i) << " : " << counts[i] << " : "
                 << counts[i] * 100.0 / total << "%" << endl;
    }

    // 3. Find the highest count
    int max = 0;
    for (int i = 0; i < 26; i++)
        if (counts[i] > max) max = counts[i];

    // 4. Treat each top letter as E and try its key
    for (int i = 0; i < 26; i++) {
        if (counts[i] == max) {
            int key = (i - 4 + 26) % 26;
            cout << "Top letter " << char('A' + i) << " -> key " << key
                 << " : " << decrypt(cipher, key) << endl;
        }
    }

    // 5. Brute force: try all 26 keys
    for (int key = 0; key < 26; key++)
        cout << "Key " << key << " : " << decrypt(cipher, key) << endl;
    return 0;
}
```

Sample run (the ciphertext is typed by the user, so nothing is hardcoded):

```
B : 3 : 12%
F : 1 : 4%
J : 5 : 20%
M : 1 : 4%
N : 1 : 4%
Q : 2 : 8%
R : 3 : 12%
T : 3 : 12%
W : 2 : 8%
Y : 4 : 16%
Top letter J -> key 5 : WE WILL MEET THE TEAM TOMORROW
Key 0 : BJ BNQQ RJJY YMJ YJFR YTRTWWTB
Key 1 : AI AMPP QIIX XLI XIEQ XSQSVVSA
Key 2 : ZH ZLOO PHHW WKH WHDP WRPRUURZ
Key 3 : YG YKNN OGGV VJG VGCO VQOQTTQY
Key 4 : XF XJMM NFFU UIF UFBN UPNPSSPX
Key 5 : WE WILL MEET THE TEAM TOMORROW
...
```

How to compile and run: save the file as freq.cpp, then use g++ freq.cpp -o freq and run ./freq (on Windows: freq.exe).

Coding Practice 2 (Solution in Pseudo Code)

Problem: Write pseudo code for a program that reads a ciphertext produced by a simple substitution cipher and prints a **partial plaintext** by matching cipher letters to English letters by frequency order (ETAOINSHRDLU). Unmatched letters must be shown as underscores (_).

```
INPUT: Ciphertext
english_order = "ETAOINSHRDLU"

// 1. Count the letters
INITIALIZE counts[A..Z] = 0
FOR each character ch in Ciphertext:
    IF ch is an alphabet:
        counts[uppercase(ch)] = counts[uppercase(ch)] + 1

// 2. Sort cipher letters by count, highest first
sorted_letters = letters with counts > 0 sorted by counts DESCENDING

// 3. Build the mapping: i-th most frequent cipher letter -> i-th English letter
INITIALIZE mapping as empty
FOR i = 0 to min(len(sorted_letters), len(english_order)) - 1:
    mapping[sorted_letters[i]] = english_order[i]

// 4. Apply the mapping
Plaintext = empty
FOR each character ch in Ciphertext:
    IF ch is an alphabet:
        IF uppercase(ch) is in mapping:
            Append mapping[uppercase(ch)] to Plaintext
        ELSE:
            Append '_' to Plaintext        // not guessed yet
    ELSE:
        Append ch to Plaintext             // keep spaces/punctuation

OUTPUT: mapping table and Plaintext
```

Tip

The output of this program is only a first guess. After running it, use small words (THE, A, I) to correct the mapping by hand, as shown in Experiment 3.